

PLAN DE DESARROLLO · APP MÓVIL DE CONTROL OPERATIVO

Ruta Falex

Falex transporta gaseosa para KR mediante trailers. Esta app reemplaza el control manual de viajes, ingresos, gastos, trabajadores, vehículos e impuestos por un registro único, filtrable por mes, trabajador o viaje — con el negocio funcionando incluso sin señal en carretera.

Cliente Falex **Preparado** 14 ago 2026 **Estado** Fase de arranque

00 Resumen ejecutivo

El objetivo es una app Flutter de uso diario para el dueño de Falex: registrar cada viaje (chofer, trailer, cliente, carga), su ingreso y sus gastos asociados, además de los movimientos generales del negocio y los impuestos por periodo. Todo debe poder filtrarse por mes, trabajador o viaje para saber, en segundos, qué tan rentable fue cada recorrido y cada mes. El desarrollo se organiza en 9 fases (0 a 8) pensadas para entregar algo usable pronto y sumar complejidad después, sin bloquear el trabajo por decisiones aún pendientes como la identidad visual.

01 Decisiones confirmadas

Base técnica ya acordada contigo. Todo lo demás en este documento se apoya en estas cinco decisiones.

FRAMEWORK

Flutter

Una sola base de código para Android e iOS; ideal para una app con muchas pantallas de formularios, listas y reportes.

DATOS

SQLite local (Drift)

Sin backend ni servidor que mantener ni facturar — costo cero, y encaja con un solo usuario en su propio teléfono.

CONECTIVIDAD

Offline-first real

Al vivir 100% en el dispositivo, la app funciona igual con o sin señal — no hay "modo offline", es el único modo.

ROLES

Un solo usuario admin

El dueño ve y edita todo. Sin login múltiple ni permisos diferenciados por ahora — simplifica toda la app.

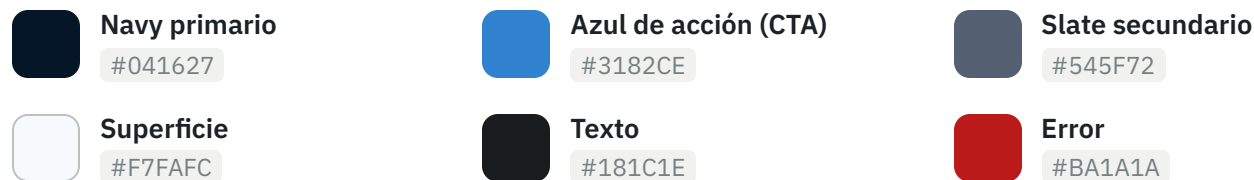
BORRADOR

Sistema visual (Stitch)

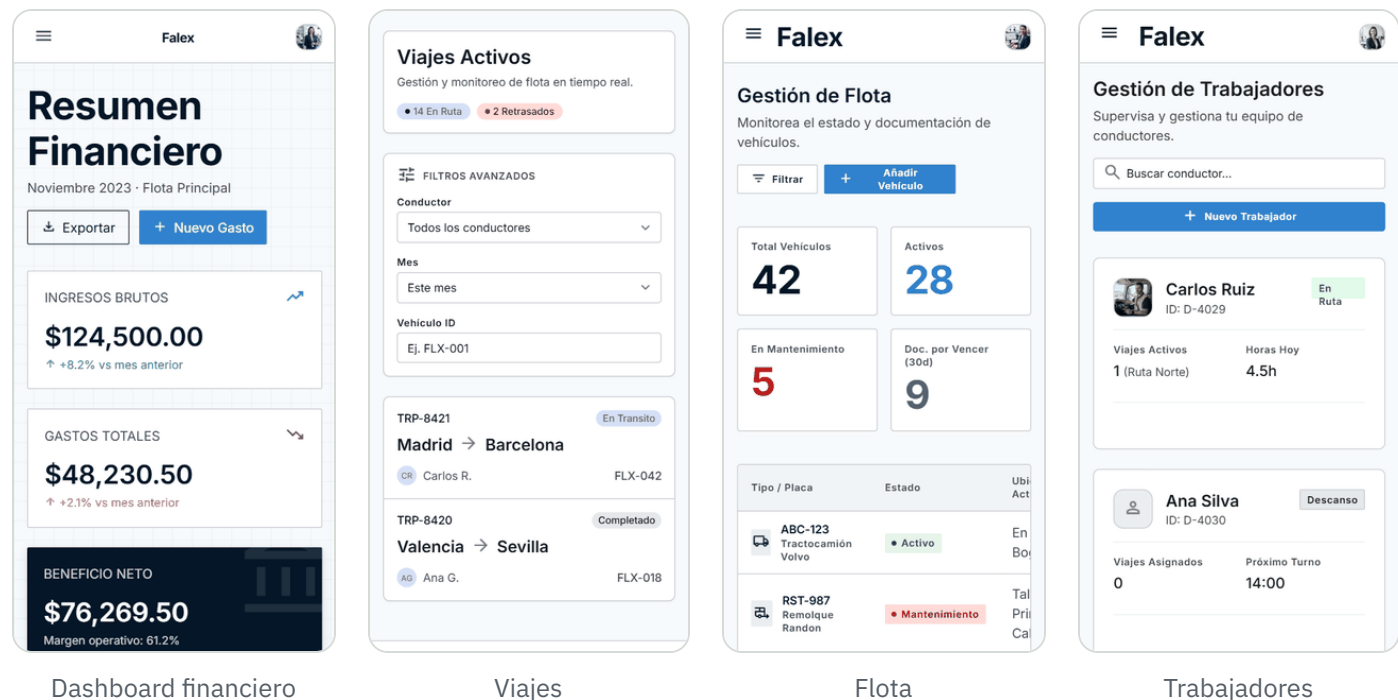
Ya hay un borrador de diseño generado en Stitch con paleta, tipografía y 4 pantallas — se detalla abajo. El logo sigue pendiente.

02 Sistema de diseño de referencia

En la carpeta del proyecto encontré `stitch_falex_operations_management_system.zip` : un borrador de UI generado con Stitch, con paleta, tipografía y 4 pantallas ya maquetadas (Dashboard financiero, Viajes, Flota, Trabajadores). No es código Flutter — es una referencia visual en HTML — pero es un punto de partida real, no genérico, y confirma varias cosas del alcance funcional: la pantalla de Viajes ya trae filtros por conductor, mes y vehículo; Flota ya trae alertas de documentos por vencer.



Tipografía **Inter** en toda la escala, radios de 4px (controles) y 8px (contenedores), grid de 4px, y profundidad por capas tonales y bordes de 1px en vez de sombras marcadas — pensado para lectura de datos densos sin fatiga.



Ojo: los datos de estas pantallas son de ejemplo (rutas Madrid–Barcelona, montos en USD) — al construir se adapta a las rutas, moneda (soles) y volumen reales de Falex. El logo que aparece es un avatar de relleno, no la marca de Falex.

Este diseño es punto de partida, no un límite.

Sirve para fijar tono, paleta y componentes — no para encerrar el alcance en las 4 pantallas que trajo. Se suman las que la app realmente necesite. Además de Dashboard, Viajes, Flota y Trabajadores, el

alcance ya pedido implica al menos estas pantallas propias que Stitch no maquetó:

- Formularios de alta/edición para cada entidad (no solo listas)
- Ingresos, Egresos e Impuestos como módulos independientes, no solo tarjetas dentro del dashboard
- Detalle de viaje con su balance (ingresos – egresos de ese viaje)
- Reportes/exportación con filtros aplicados
- Estados vacíos y de error (sin sacar la app "en blanco" cuando algo falla o no hay datos aún)
- Bloqueo con PIN/huella y pantalla de respaldo/restauración

03 Alcance funcional

Ocho módulos cubren el pedido original: viajes, trabajadores, vehículos, ingresos, egresos e impuestos, con filtros y un dashboard que amarra todo.

Viajes

Registro central: fecha, origen/destino, cliente, chofer, trailer, carga y estado del recorrido.

Trabajadores

Choferes y personal: datos, cargo, estado activo/inactivo, historial de viajes asignados.

Vehículos / trailers

Ficha por unidad con documentos (SOAT, revisión técnica) y estado operativo.

Ingresos

Fletes cobrados a KR y otros ingresos, ligados o no a un viaje puntual.

Egresos / gastos

Combustible, peajes, viáticos, mantenimiento, multas — por categoría, con foto del comprobante.

Impuestos

Obligaciones por periodo, monto, vencimiento y estado de pago, con comprobante adjunto.

Filtros y reportes

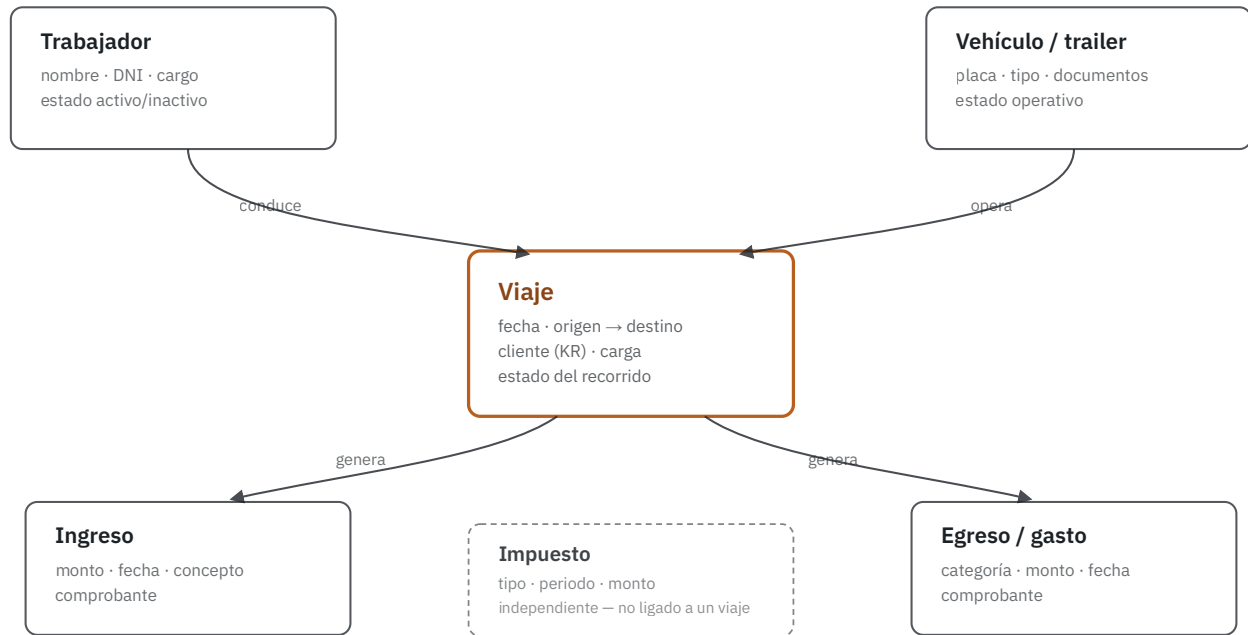
Por mes, trabajador, vehículo o viaje — con exportación a PDF/Excel para el contador.

Dashboard

Balance del mes, viajes activos y próximos vencimientos, de un vistazo al abrir la app.

04 Modelo de datos

El **Viaje** es la entidad bisagra: conecta a quién manejó, en qué unidad, y qué dinero entró y salió por ese recorrido. Impuestos vive aparte porque se paga por periodo, no por viaje.



Trabajador y Vehículo se asignan a un Viaje; cada Viaje genera sus propios Ingresos y Egresos. Impuesto queda fuera de esa cadena: se administra por periodo fiscal, no por recorrido.

05 Arquitectura técnica

Cuatro capas simples, sin servidor de por medio. Cada pantalla habla con un repositorio, nunca directo con la base de datos.

UI Pantallas Flutter widgets por feature: viajes, trabajadores, vehículos, ingresos, egresos, impuestos.	ESTADO Riverpod Providers que exponen datos y acciones a la UI, sin lógica de base de datos dentro de las pantallas.	DOMINIO Repositorios Un repositorio por entidad; aísla a la app del motor de datos y facilita testear.	DATOS Drift (SQLite) Base de datos local en el propio teléfono, con migraciones versionadas desde el primer commit.
---	---	---	--

Respaldo: exportación manual de la base de datos a Google Drive del dueño (cuenta gratuita, sin costo de infraestructura). Si más adelante se necesita que varias personas vean la información en tiempo real desde distintos dispositivos, el punto natural de migración es Firebase (capa gratuita) o Supabase — pero no se justifica para un solo usuario offline-first.

06 Validaciones y manejo de errores

Cada entidad necesita reglas claras antes de escribirse en la base de datos, y cada pantalla necesita un plan para cuando algo sale mal — no solo para el camino feliz. Esto no es opcional ni se deja para el final: se construye módulo a módulo, junto con cada CRUD de la fase que le corresponde.

Principios generales

- **Nunca borrado físico** de Trabajador o Vehículo — solo baja lógica, porque tienen historial de viajes ligado
- **Confirmación explícita** antes de cualquier acción destructiva (eliminar gasto, cancelar viaje, restaurar backup) — en SQLite local no hay "deshacer" real
- **No se puede inactivar** un trabajador o vehículo con viajes en curso sin avisar primero
- **Guardado fallido** (poco común, ej. disco lleno) muestra el error real y conserva lo que el usuario ya escribió — nunca se pierde el formulario
- **Estados vacíos** con mensaje útil ("Aún no registras viajes este mes") en vez de pantallas en blanco

ENTIDAD	REGLA CLAVE	QUÉ EVITA
Trabajador	DNI único, 8 dígitos	Duplicados y datos mal tipeados
Vehículo	Placa única, formato válido	Confundir dos unidades en un reporte
Viaje	Fecha salida ≤ fecha llegada; chofer y vehículo deben estar "activos"	Asignar el viaje a alguien de baja o una unidad en mantenimiento
Ingreso / Egreso	Monto > 0, máx. 2 decimales; fecha dentro de un rango razonable	Balances rotos por errores de tipeo (ej. 1500000 en vez de 150.00)
Impuesto	Fecha de vencimiento obligatoria y posterior a la fecha de registro	Recordatorios que nunca se disparan o llegan tarde
Adjuntos	Tamaño máximo por imagen, con compresión automática	Base de datos local pesada y app lenta tras meses de uso

07 Funcionalidades que suman valor

Más allá del CRUD básico, esto es lo que hace que el dueño realmente abra la app todos los días en vez de volver al cuaderno. Ninguna es imprescindible para el MVP, pero todas son de bajo esfuerzo si se piensan desde la fase indicada.

FUNCIONALIDAD	POR QUÉ AYUDA	FASE SUGERIDA
Duplicar viaje	Las rutas para KR se repiten seguido; copia chofer, vehículo y cliente, y solo pide la fecha nueva	Fase 3
Registro rápido de gasto	Anotar el peaje o el combustible en el momento, sin navegar varias pantallas	Fase 4
Papelera con restaurar	Un borrado accidental de un viaje o gasto no debería ser definitivo al toque	Fase 4
Alertas de presupuesto	Avisar si "Combustible" del mes ya superó el promedio de meses anteriores	Fase 6
Comparativa mes contra mes	Ver si el mes fue mejor o peor que el anterior, no solo el total aislado	Fase 6
Exportar y compartir por WhatsApp	Enviar el PDF del reporte directo al contador sin pasar por correo	Fase 6
Búsqueda global	Buscar un viaje, trabajador o comprobante por texto libre, sin recordar en qué mes fue	Fase 6
Modo oscuro	Uso frecuente en carretera de noche o con poca luz	Fase 7
Acceso directo "Nuevo gasto"	Reduce la fricción para registrar en el momento y no al final del día, cuando ya se olvidó	Fase 7

08 Roadmap por fases

Nueve fases, de la Fase 0 (arranque) a la Fase 8 (publicación). Cada una entrega algo verificable antes de sumar la siguiente.

0

Fundamentos y arranque Semana 1

Dejar listo el entorno y las reglas del proyecto antes de tocar una pantalla.

- ☐ Confirmar con el dueño de Falex qué va en el MVP y qué queda para después
- ☐ Levantar el proyecto Flutter con estructura por feature (viajes, trabajadores, vehículos, ingresos, egresos, impuestos, core)
- ☐ Configurar linter, formatter y convención de commits
- ☐ Elegir Riverpod como gestor de estado y dejar las capas UI → Providers → Repositorios → Datos
- ☐ Portar el borrador de Stitch (navy, azul de acción, tipografía, radios 4/8px) a un `ThemeData` centralizado en Flutter

1

Modelo de datos y base local Semana 2

Construir el esqueleto de datos que sostiene toda la app.

- ☐ Modelar entidades definitivas: Trabajador, Vehículo, Viaje, Ingreso, Egreso, Impuesto, Adjunto
- ☐ Definir con el dueño las categorías reales de gasto (combustible, peajes, viáticos, mantenimiento, multas, otros)
- ☐ Definir los tributos aplicables al régimen de Falex (para modelar Impuesto correctamente)
- ☐ Base de datos local con Drift y migraciones versionadas desde el primer esquema
- ☐ Un repositorio por entidad, con datos de prueba (seed) para desarrollar sin datos reales

2

Catálogos base: trabajadores y vehículos Semana 3

Tener listos los "maestros" de los que depende todo lo demás.

- ☐ CRUD de trabajadores, con baja lógica (nunca borrar para no perder historial de viajes)
- ☐ CRUD de vehículos/trailers, con vencimientos de SOAT y revisión técnica
- ☐ Listas con búsqueda y filtro por estado activo/inactivo
- ☐ Validaciones: placa única, DNI único

3

Viajes — el núcleo del negocio Semanas 4-5

Registrar cada viaje como la unidad que conecta trabajador, vehículo y dinero.

- ☐ Alta de viaje: fecha, origen/destino, cliente, chofer, trailer, carga, kilometraje
- ☐ Estados del viaje: programado, en curso, finalizado, cancelado
- ☐ Detalle de viaje que agrupa sus ingresos y egresos asociados, con balance del viaje
- ☐ Registro rápido de gasto desde la propia pantalla del viaje, pensado para usarse en ruta

4

Ingresos y egresos generales Semana 6

Cubrir el dinero que no está atado a un viaje puntual.

- ☐ CRUD de ingresos, ligados o no a un viaje
- ☐ CRUD de egresos por categoría, ligados o no a un viaje o a un vehículo (mantenimiento)
- ☐ Foto de boleta/factura por cámara o galería en cada movimiento
- ☐ Balance automático por viaje, por mes y general

5

Impuestos Semana 7

Llevar el control tributario dentro de la misma app.

- ☐ CRUD de impuestos por periodo: tipo, monto, vencimiento, estado de pago
- ☐ Recordatorio local (notificación) cerca de la fecha de vencimiento
- ☐ Comprobante de pago adjunto

6

Dashboard, filtros y reportes Semanas 8-9

Convertir el registro diario en información para decidir.

- ☐ Dashboard: balance del mes, viajes activos, próximos vencimientos
- ☐ Filtros combinables por mes, trabajador, vehículo o viaje
- ☐ Exportar reporte filtrado a PDF/Excel para el contador o el dueño
- ☐ Gráfico simple de tendencia mensual: ingresos, egresos, utilidad

7

Seguridad, respaldo y pulido Semana 10

Proteger la información financiera y evitar perder datos.

- ☐ Bloqueo de la app con PIN o huella, por tratarse de datos financieros en el teléfono
- ☐ Backup manual de la base de datos a Google Drive del dueño
- ☐ Restaurar backup desde archivo
- ☐ Validar con el dueño el sistema visual de Stitch (o ajustarlo) e incorporar el logo definitivo sobre el theme ya centralizado

8

QA y publicación Semanas 11-12

Validar el flujo real antes de que el negocio dependa de la app a diario.

- ☐ Prueba con datos reales de un mes completo de operación
- ☐ Prueba offline real: modo avión, sin señal en ruta, backup al recuperar conexión
- ☐ APK firmado para instalación directa (o publicación interna en Play Store)
- ☐ Capacitación breve al dueño sobre el uso diario de la app

09 Backlog futuro y riesgos

Fuera del alcance inicial

- **Multi-usuario con roles** — choferes registrando su propio viaje, dueño aprobando
- **Sincronización multi-dispositivo** o panel web para el contador
- **Notificaciones push** de vencimientos, más allá de las locales
- **Integración con SUNAT** o facturación electrónica

Riesgos a vigilar

- **Alcance amplio** — conviene cerrar un MVP real (Fases 0-4) antes de sumar impuestos y reportes
- **Categorías tributarias** deben definirse con el dueño antes de la Fase 5; varían según régimen
- **Backup manual** depende de que el dueño lo ejecute — evaluar automatizarlo por calendario
- **Logo aún pendiente** — el borrador de Stitch trae paleta y tipografía, pero solo un avatar de relleno en vez de marca

10 Próximos pasos inmediatos

Lo que arranca esta misma semana, antes de escribir la primera pantalla.

- 1 Confirmar este plan y el orden de fases con el dueño de Falex
- 2 Crear la estructura del proyecto Flutter dentro del repo (`app-falex/app-falex`)
- 3 Definir con el dueño las entidades finales y las categorías de gasto e impuesto
- 4 Instalar y dejar configurado Riverpod como gestor de estado
- 5 Armar el esquema inicial de Drift descrito en la Fase 1

Documento vivo: se actualiza a medida que avanzan las fases y se toman nuevas decisiones con el dueño de Falex.

El sistema visual parte del borrador de Stitch encontrado en el repo; el logo definitivo queda fuera de este plan a propósito — se define más adelante sin afectar el cronograma.